



INTRODUCTION TO ADVANCED ROBOTICS

PROJECT 2

FINDING THE BOARD

WHAT'S NEXT?

In Project 1 you built something that could name a shape. It was a good program, and it had one very specific blind spot: it could tell you **what** something was, but never **where** it was.

That is enough to print a word on screen. It is not enough to move an arm. To pick something up you need to know how far away it is and which way to turn — numbers, not nouns.

So this project changes what the program looks for. Instead of hunting for shapes, it hunts for **markers** you have taped to the board, works out exactly where each one is in three dimensions, and prints the numbers in **millimetres**. Everything you already know carries over. Imports, the robot startup, the camera scan and the display loop are all still here. What changes is the middle of the program, where the shape maths used to be.

TWO SQUARES, ONE ANSWER

Put a large square on the left of the plate and a small square on the right. Both of them measure an extent of about 0.99, both score well on circularity, both have four corners. Your Project 1 decision chain would call them *Square* — the same word, for two things in different places.

The decision chain was never broken. It was answering the only question it had been given. **Shape answers what. A machine that moves things needs where.**

This project supplies the missing half. You already know what the things on your board look like, because you put them there. So the computer has something it can

- *A measurement you can repeat is worth more than a clever guess. You already know what the things on your board look like, because you put them there.*

BEFORE YOU TYPE ANYTHING

The very first line of the starter file is not Python. It is an instruction to a human:

```
# Remember to cover the two markers on the board for this project!
```

Two printed ArUco markers are taped to the board, and the camera looks down at them from above. Cover them when you are not using them.

The reason is simple. A marker the camera can see is a marker the program will measure, and a marker left on the board is a marker your arm may try to reach. You will be sending this arm to real coordinates later in the module.

Check three things before you start: the markers are flat and not curled, the camera can see both of them at once, and you have measured one marker edge with a ruler. That measurement becomes a number in the program, and every distance it prints depends on it. If an image you take hides a marker or it is read wrongly, or ignored.

WHAT IS AN ARUCO MARKER?

A square of pure black and pure white: a thick black border all the way round, and a grid of black and white squares inside it.

The border is identical on every marker ever made. The grid inside is the part that carries the identity. The program already knows what all 250 valid grids look like, so when it sees one it can say which marker it is just by looking.

This is what computer people call a **fiducial** — an ordinary object placed in the scene purely so the computer has something known to measure against. The marker is not the subject of the picture. It is the ruler. The properties make this work in a real classroom. Markers are cheap to print and they are designed so the black border cannot be confused with the grid inside it, which is what lets the detector find a marker at an angle instead of only face-on.

DICT_6X6_250

```
self.aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
```

6X6 means the inner grid is six squares across and six down. 250 means there are 250 different valid patterns in this dictionary.

Why a dictionary at all? Because the marker has to be unambiguous. The detector must never mistake a row of payload squares for the border, and no two markers in the dictionary may look alike from any angle.

Changing the dictionary is therefore a decision with consequences: it decides which physical markers your program is willing to recognise. Change it without changing the markers on the board and the program will report nothing at all — with no error message to explain why. The detector also needs the marker's known size in millimetres, from your earlier ruler measurement.

THE CAMERA IS NOT PERFECT

```
self.camera_matrix = np.array([
    [781.33379113, 0.0, 347.53500524],
    [0.0, 783.79074192, 246.67627253],
    [0.0, 0.0, 1.0]
], dtype=np.float32)
```

781.33 and 783.79 are **focal lengths**, measured in pixels. They say how far the lens "sees", in the same units as the picture itself.

347.53 and 246.67 are the **principal point** — the pixel where the camera's own line of sight lands on the sensor. Usually near the middle of the picture, but not exactly.

The bottom row of zeros and a one is bookkeeping, not a measurement. It is there because matrix maths expects this shape.

Produced from a checkerboard photo, they belong to this exact camera and lens. Copy them exactly.

AND THE LENS IS NOT FLAT

```
self.dist_coeffs = np.array([
    [3.41360787e-01, -2.52114260e+00, -1.28012469e-03, 6.70503562e-03, 2.57018000e+00]
], dtype=np.float32)
```

Five numbers describing how badly this particular lens bends light. Cheap lenses bend more towards the edges than in the middle, so a square photographed near the corner of the picture comes out slightly rounded.

They are written in scientific notation, which is just a compact way of saying a very small or very large number. $3.41\text{e}-01$ is 0.341. $-2.52\text{e}+00$ is -2.52.

Without these five numbers every distance would be slightly wrong — and wrong by a **different amount** depending on where in the picture the marker happened to be sitting. A marker in the centre and the same marker in the corner would give two different answers.

PIXELS ARE NOT A DISTANCE

```
# ArUco marker size in meters (30mm)
self.marker_size_m = 0.03
```

Here is the whole trick. The camera hands you a marker as **pixels**, and pixels have no units. You cannot move an arm by an amount of pixels.

So you give the program one real measurement — how wide the marker actually is — and it works backwards from the marker's size on screen to how far away it must be.

Measure it once, with a ruler, carefully. `0.03` metres is 30 millimetres, which is why the comment spells it out: a `base` — it would mean three kilometres. ~~base is the only measurement your program trusts.~~ If it is wrong, nothing downstream can detect it — the program has no way to know that the marker was printed at 29mm instead of 30mm, and it will report confident, tidy, incorrect distances forever.

ASKING WHERE IT IS

```
rvecs, tvecs, _ = cv2.aruco.estimatePoseSingleMarkers(  
    corners, self.marker_size_m, self.camera_matrix, self.dist_coeffs  
)
```

One line, three inputs, two answers. Corners in pixels, the marker's real size, and the two calibration arrays go in; a rotation and a position come out — for every marker in the picture at once.

`rvecs` is rotation. `tvecs` is translation. The third return value is thrown away here, which is what the underscore is for.

The word **pose** means both halves at once: where the marker is *and* which way it is facing. For sorting shapes you could ignore the second half. For a robot that has to line up a gripper, you cannot. `estimatePoseSingleMarkers` has its own — the right choice when markers move independently.

THREE NUMBERS: X, Y, Z

```
tvec = tvecs[i][0] # [X, Y, Z] in meters
```

`tvecs[i]` is the translation of marker number `i`. The `[0]` unwraps it down to a plain array of three numbers, which is much easier to work with.

Two of the three describe sideways movement and the third is **depth** — how far away the marker is. All three are in metres, and all three are measured from the camera, not from the robot.

A negative number is not an error. It simply means the marker is to the left, or below the camera's own origin, or nearer than the camera itself. The signs carry real information about where things are.

The loop that follows goes round `for i in range(len(ids))` — one pass per marker found. Everything inside that loop is about marker number `i`, and nothing inside it should assume there is more than one marker.

THE ROTATION YOU CANNOT READ

`rvecs[i]` is also three numbers, and unlike `tvecs[i]` they are not coordinates. A rotation vector packs two separate ideas into three values.

Its **length** is the angle turned, in radians. Its **direction** is the axis you turned about. Together they say "tip this far, around this line".

OpenCV represents it this way because it composes cleanly: add two rotation vectors and you get another valid rotation. Three separate Euler angles do not behave that way, and roll over.

- *Notice what the program does with it: nothing. `rvecs[i]` is handed straight to the drawing function, which does the trigonometry and draws the answer. A library doing the hard arithmetic is the whole point of using a library.*

WHOSE COORDINATES?

Everything so far is measured **from the camera**. But the arm does not live inside the camera. The robot has no idea where the lens is, and moving to a camera-relative coordinate would put the gripper in completely the wrong place.

So there are two frames in this program, and one conversion between them. This is the single most

What	Camera frame	End-effector frame
Measured from	The lens	The pump on the wrist
Unit	metres	millimetres
X and Y mean	sideways on screen	sideways from the wrist
Z means	depth from the lens	depth from the wrist
Used by	the maths library	the arm

A coordinate is only ever meaningful once you have said **relative to what**. Two systems using the same three numbers and different origins will send an arm to two completely different places.

MOVING FROM ONE FRAME TO THE OTHER

```
# Pump / End-effector spatial offsets (mm)
self.pump_x = 15
self.pump_y = -55
```

```
rel_x = round(tvec[0] * 1000 + self.pump_y, 2)
rel_y = round(tvec[1] * 1000 + self.pump_x, 2)
rel_z = round(tvec[2] * 1000, 2)
```

The `* 1000` is the unit change — metres become millimetres. The two offsets are the measured gap between where the camera thinks it is and where the pump really is, and they are added in to correct for it.

Now read those two lines carefully, because they will catch you out. The value named `pump_y` is added to `x`, and `pump_x` is added to `y`. The names are crossed over.

- *Do not "fix" it — the crossing is deliberate: the camera axes run the other way. Check one known marker if you change these.*

THREE OVERLAYS, THREE JOBS

```
cv2.aruco.drawDetectedMarkers(frame, corners, ids)
```

```
cv2.drawFrameAxes(  
    frame, self.camera_matrix, self.dist_coeffs,  
    rvecs[i], tvecs[i], self.marker_size_m * 0.5  
)
```

```
cv2.putText(  
    frame, label_text, (corner_center[0] - 80, corner_center[1] - 15),  
    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2  
)
```

`drawDetectedMarkers` draws the outline and the ID number.

`drawFrameAxes` stands a small RGB tripod on the marker showing which way it faces. `marker_size_m * 0.5` keeps it in proportion at any range.

`putText` writes your numbers on the picture. `(0, 255, 0)` is green — OpenCV lists colour channels blue, green, red.

TWO VERSIONS OF THE SAME LIBRARY

```
try:
    self.aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters()
    self.detector = cv2.aruco.ArucoDetector(self.aruco_dict, self.aruco_params)
    self.use_new_aruco = True
except AttributeError:
    self.aruco_dict = cv2.aruco.Dictionary_get(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters_create()
    self.use_new_aruco = False
```

OpenCV moved the ArUco API: newer builds give you a detector object to keep; older builds pass a dictionary each call.

The `except AttributeError` catches one situation: this version has no such name — a missing attribute, not a crash.

`self.use_new_aruco` is set once here and read each frame to pick which call style to use. Detect the version once, then branch cheaply in the loop.

A pattern worth copying: four lines and the same file runs on the classroom laptop and the arm, even with different OpenCV versions.

THE WHOLE PIPELINE ON ONE LINE

```
display_frame, log_info = vision.process_frame(frame)
```

That single call, once per frame, does all of this:

- 1 Grey the picture**
ArUco detection works on greyscale, not colour.
- 2 Detect the markers**
Find every corner of every marker in view.
- 3 Draw the outlines**
So you can see what it found.
- 4 Estimate the pose**
Pixels plus marker size become metres and radians.
- 5 Convert frames**
Camera metres to wrist millimetres.

It returns two things: the picture with the overlays drawn on it, and a line of text for the terminal. One for your eyes, one for your log.

- 6 Draw axes and text**
Put the answer on the picture.

STEP 1

The first TODO sits at the very top of the file, before anything else can possibly work. Three packages, in this order:

```
import time
import cv2
import numpy as np
```

Order does not matter here, but do not leave one out. `time` gives you the pauses that let the arm finish a movement, `cv2` is OpenCV, and `numpy` holds the calibration arrays.

The next TODO imports `MyCobot280` from `pymycobot`. That one is not in any standard library, so it is the import most likely to fail on a machine that has never run the arm before.

STEP 2

The robot import lives on its own line:

```
from pymycobot.mycobot280 import MyCobot280
```

This reads as: from the `mycobot280` part of the `pymycobot` package, bring in the `MyCobot280` class. Importing a class rather than a whole module means you can type the shorter name afterwards.

If this line fails, the package is not installed on that machine. That is a different problem from a missing import, and the error message will tell you which one you have. The rest of the deck assumes this import works. On a laptop that has never had the arm connected, expect to see it fail and be told to skip to Step 5 — the vision pipeline does not need the robot at all.

STEP 3

This whole section sits inside a `try` block, which is deliberate. Watch what each line does:

```
print("Connecting to myCobot280...")
mc = MyCobot280('/dev/ttyAMA0', 1000000)
time.sleep(0.5)
mc.power_on()
time.sleep(0.5)
```

The port `/dev/ttyAMA0` is which serial socket the arm is plugged into, and `1000000` is the speed both ends agree to talk at. On another machine this may be `ttUSB0`, so if this line throws, check the port before you suspect the arm.

Each `sleep` gives the command before it time to finish. Delete them and the arm misses instructions, which looks exactly like a broken arm.

If the arm is not connected, this block prints `Robot hardware warning:` and carries on. That is the design working, not a failure you need to fix — the vision pipeline runs perfectly well without hardware.

STEP 4

Park the arm somewhere safe before the camera work starts:

```
folded_angles = [0, 45, -90, -45, 0, 0]
print("Moving arm to initial folded position...")
mc.send_angles(folded_angles, 20)
time.sleep(2.0)
```

Six numbers, one per joint, in degrees. Zero means `no bend`, and the signs say which way. Folding the arm up and out of shot is deliberate: a metal arm parked in view of the camera is a large shiny object that will confuse everything you measure.

The `20` is how fast to travel there, and the `2.0` second sleep gives it time to arrive before the camera starts.

Notice this is a `list`, not a `tuple`. Later in the module you will want to change these values while the program runs, and the `[ ]` brackets are what allow that.

RUN IT BEFORE YOU FIX ANYTHING

Send the starter across and run it with the four vision TODOs still empty. Seeing the failures first is worth the two minutes.

```
scp M2-P2-Starter.py er@192.168.1.149:Documents
```

```
python M2-P2-Starter.py
```

The robot block will warn and continue. Then the camera opens, a frame arrives, and `process_frame` runs with the detection code missing. Expect:

```
NameError: name 'ids' is not defined
```

Read that message properly. It is not a camera fault and it is not a marker fault — it says a variable the program expected to have been created was never created. That is Step 6, and nothing after it.

STEP 5

Now we move into the class. This TODO asks for the dictionary and the detector — the version-tolerant block from the concepts section:

```
try:
    self.aruco_dict = cv2.aruco.getPredefinedDictionary(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters()
    self.detector = cv2.aruco.ArucoDetector(self.aruco_dict, self.aruco_params)
    self.use_new_aruco = True
except AttributeError:
    self.aruco_dict = cv2.aruco.Dictionary_get(cv2.aruco.DICT_6X6_250)
    self.aruco_params = cv2.aruco.DetectorParameters_create()
    self.use_new_aruco = False
```

This runs once, when the object is created — not once per frame. Read it as setup, not as part of the loop.

If you want proof it worked, print `self.use_new_aruco` after the block. It tells you which half of the `try` your machine took, and that is genuinely useful to know when something downstream misbehaves.

STEP 6

This is the TODO the error message was pointing at. Detect the markers, choosing the call style the flag decided on:

```
if self.use_new_aruco:
    corners, ids, _ = self.detector.detectMarkers(gray)
else:
    corners, ids, _ = cv2.aruco.detectMarkers(
        gray, self.aruco_dict, parameters=self.aruco_params
    )
```

The greyscale conversion happens just above this block, on its own line. ArUco detection works on brightness, not colour.

Three things come back. `corners` holds four points per marker, `ids` which marker each one was, and the third is rejected. When nothing is found, `ids` is `None`, not an empty list — which is why the next line tests both.

Once this works, the terminal stops shouting and the window appears — still blank, since nothing is drawn yet. Correct progress.

STEP 7

Detection found something. Now put it on the screen:

```
cv2.aruco.drawDetectedMarkers(frame, corners, ids)
```

One line, no configuration, no return value. It draws an outline around each marker and stamps its ID number beside it.

Look closely at what it did *not* do: the outlines have no depth. Nothing on screen yet tells you whether the marker is 200mm away or 800mm — this is still a flat picture.

Run the program now. A coloured rectangle with a number on it is the single best checkpoint in this project: it proves the camera, the dictionary, the lighting and your marker printing are all fine, and it narrows every later fault to the maths that remains.

STEP 8

Now the hard part. One call turns corners into a position and a tilt:

```
rvecs, tvecs, _ = cv2.aruco.estimatePoseSingleMarkers(  
    corners, self.marker_size_m, self.camera_matrix, self.dist_coeffs  
)
```

Every input here was prepared earlier in the file: `corners` from Step 6, `marker_size_m` from your ruler, and the two calibration arrays that were already written in for you.

This call is where the mathematics actually happens. It undoes the lens distortion, works out how far away a marker of that known size must be, and works out which way it is tilted — all from four corners.

Four numbers in, two sets of three numbers out, for every marker at once. Nothing here is approximate and nothing is random: the same four corners always give the same answer.

STEP 9

This is the conversion from the camera's world to the robot's:

```
rel_x = round(tvec[0] * 1000 + self.pump_y, 2)
rel_y = round(tvec[1] * 1000 + self.pump_x, 2)
rel_z = round(tvec[2] * 1000, 2)
```

Three operations per axis. Unwrap the array, convert metres to millimetres, add the offset that accounts for the pump not being exactly where the camera is, then round to two decimal places.

These are the first numbers in the whole program that the **arm** could act on. Everything before this point was maths for the sake of maths; from here on, a wrong number means the gripper goes somewhere real.

Cross-check before you trust them: cover one marker and watch the other's numbers. Then move a marker a known distance — 100mm further away — and confirm Z changes by about 100. A unit error shows up immediately at this scale.

STEP 10

The numbers are correct but invisible. This draws them:

```
cv2.drawFrameAxes(  
    frame, self.camera_matrix, self.dist_coeffs,  
    rvecs[i], tvecs[i], self.marker_size_m * 0.5
```

A small tripod of coloured lines standing on the marker: red for one axis, green for another, blue for the third. Where the lines point is which way the marker is facing.

This is the `rvecs[i]` you never had to understand, made visible. Half the marker size keeps the tripod in proportion whether the marker is near or far.

The axes are also a sanity check. If they point sideways while the marker is clearly lying flat on the board, your rotation is being interpreted in a different order than you assumed — check the axes before trusting any number printed beside them.

STEP 11

Last of the drawing. Work out where to put the text, build the label, and write it:

```
corner_center = np.mean(corners[i][0], axis=0).astype(int)
label_text = f"ID:{marker_id} X:{rel_x} Y:{rel_y} Z:{rel_z}mm"
```

```
cv2.putText(
    frame, label_text, (corner_center[0] - 80, corner_center[1] - 15),
    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2
)
```

`np.mean` averages the four corners into the middle of the marker, and `astype(int)` turns the result into whole numbers, because OpenCV will not draw text at a fractional pixel position.

The `f"..."` is an f-string: the letters stay as they are and each `{...}` is replaced by the value of the variable inside it. Notice `mm` sits outside the braces — that is literal text, not a variable.

STEP 12

Down near the bottom of the file, outside the class. This one finds a camera that actually works:

```
def find_working_camera(max_tests=5):  
    """Scans video indices and returns the first camera that produces frames."""  
    for idx in range(max_tests):  
        cap = cv2.VideoCapture(idx, cv2.CAP_V4L2)  
        if cap.isOpened():  
            ret, frame = cap.read()  
            cap.release()  
            if ret and frame is not None and frame.size > 0:  
                print(f"Found active camera at index {idx}")  
                return idx  
  
    return None
```

Opening a camera successfully does not mean it works. `isOpened` only reports that the handle was created, so the code goes one step further and tries to read an actual frame from it.

Note the `cap.release()` before the frame check. Every failed attempt is closed as it is rejected — otherwise a five-camera scan leaves four cameras held open and the good one unable to start.

STEP 13

The last big TODO wraps the whole camera loop. Most of it is already written; three lines are yours.

```
cap_num = find_working_camera()
if cap_num is None:
    print("Error: No working camera stream found.")
    return
```

```
cap = cv2.VideoCapture(cap_num, cv2.CAP_V4L2)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
```

The `is None` test asks whether the scan found anything at all, and `return` leaves the whole function immediately rather than carrying on with a camera that does not exist.

Note the wording of the two `cap.set` lines: you are asking, not telling. Some cameras ignore the request, so print `cap.get(cv2.CAP_PROP_FRAME_WIDTH)` if the window looks wrong. This function runs once, and `while True` below it runs forever. Anything inside the loop happens thirty times a second, so keep it cheap.

STEP 14

Inside the loop, the call that ties the whole class together:

```
display_frame, log_info = vision.process_frame(frame)
```

Two return values, unpacked into two names.

`display_frame` is the picture with everything drawn on it;

`log_info` is the line of text for the terminal.

Both names are worth keeping. The picture is for your eyes and the text is for your log — and the text is far easier to copy into a spreadsheet than anything you could read off a screenshot.

```
if "Searching" not in log_info and "No Frame" not in log_info:  
    print(f"\r[Detected] {log_info}", end="")
```

This next block is already written, and it is the first thing that reads `log_info`. It deliberately stays quiet while the program is still searching, so the terminal is not flooded with a new line per frame. The `\r` and `end=""` are what turn sixty prints a second into one updating line.

STEP 15

The last TODO. Without it the program runs perfectly and shows you nothing at all:

```
if display_frame is not None and display_frame.size > 0:  
    cv2.imshow("ArUco Marker Pose Detection", display_frame)
```

The window title is the first thing a beginner searches for when the program appears to do nothing, so give it a name that says what is inside.

The guard above it is not decoration. A camera that has just been unplugged can hand back an empty frame, and `imshow` on an empty frame throws on some systems.

```
if cv2.waitKey(1) & 0xFF == ord('q'):  
    break
```

`waitKey(1)` waits one millisecond and returns the key that was pressed. `& 0xFF` masks off the extra bits some platforms report, and `ord('q')` turns the letter into the number the comparison needs.

READING THE OUTPUT

```
status_msg = f"ID: {marker_id} | X: {rel_x}mm | Y: {rel_y}mm | Z: {rel_z}mm"
```

One line per marker, per frame. Example:

```
ID: 0 | X: -142.5mm | Y: 88.0mm | Z: 415.7mm
```

Break it into the four things it tells you. The `ID` is which physical marker you are looking at. `X` and `Y` are sideways offsets from the pump, and `Z` is how far in front of the pump it is.

While the camera sees nothing, the line reads `Searching for markers...` instead. That is your cue to check lighting, focus and marker condition — not to change any code.

The numbers jitter slightly frame to frame — sensor noise, not a fault. You want a reading that stays roughly steady with your hand off the board, and moves smoothly and predictably when you move a marker.

BEFORE YOU TUNE ANYTHING

Three faults account for almost every bad reading. Learn to recognise them from the symptom rather than changing code at random.

Symptom	Most likely cause	Check
Never finds a marker	Markers too dark, blurred or too small on screen	Lighting, focus, distance
Finds then loses it repeatedly	Glare or reflection on the marker	Matte card, change the angle of the light
Numbers jump when still	Marker not flat, or board not rigid	Re-tape it, flatten the board
Numbers roughly right, all too large	marker_size_m too small	Re-measure the marker edge
Numbers drift as the arm moves	Vibration reaching the camera	Wait for the arm to settle before reading

Notice how four of those five are physical problems with the setup rather than faults in the program. Before you change a line of code, ask whether the camera can physically see what it is being asked to see.



EXTENSION 1

MAKE THE ARM AIM AT THE MARKER

THE BRIEF

You have a program that measures a marker and prints where it is. A robot needs to do something with that number.

Close the loop. Read the marker position, convert it into joint angles, and let the arm move towards it — then find out how accurate your measurement actually was.

- *Accuracy is the whole point. You are about to find out how far a 30mm ruler measurement and a real camera agree, and the answer is more interesting than you expect.*

YOUR MISSION: AIM AT THE MARKER

Turn a printed number into a movement, then measure how wrong you were.

- 1 Move the arm to the marker**
Use the X, Y and Z you already print to build a set of joint angles and send the arm there. Start with a small movement — this arm is real hardware and the marker size on screen is not the marker size in the room.
- 2 Prove the reading is repeatable**
Twenty readings from a marker that has not moved. Record the spread. Then move the marker a known 100mm and check that Z changes by 100. If it does not, your error is in the calibration, not in the maths.
- 3 Change the rule**
One marker is easy. Try a two-marker board and aim at whichever you moved, or use distance to trigger a pick when a shape sits in place.
- 4 Hand it over**
Write the handover page: your marker size, your measured accuracy, what you would recalibrate, and one safety limit for the next person.

- *Implement aiming by changing arithmetic, not by re-teaching the arm a new pose each time. A table of recorded angles solves the wrong problem.*

YOUR DATA LOG

Write down what you measured before you tune anything. Every number in this project depends on one ruler measurement.

Marker ID	X (mm)	Y (mm)	Z (mm)	My values
0, marker still				
0, moved 100mm				
1, marker still				
1, moved 100mm				

Then write your aiming rule as a set of conditions before you code it:

IF Z is _____ THEN move to _____
IF Z is _____ THEN move to _____
OTHERWISE _____

● *A rule you cannot say out loud is a rule you have not decided yet.*

DEFINITION OF DONE

- ☐ Both markers detected, outlined and numbered, from three different positions on the board
- ☐ Axis tripod and coordinate text drawn on every detected marker
- ☐ Your measured marker size recorded in millimetres, with the ruler reading it came from
- ☐ Twenty repeat readings logged, and the spread written down
- ☐ Arm moved to a marker position, and the achieved accuracy compared against the reading you aimed with

GO FURTHER

- Average several frames. Pose estimates jitter by a millimetre or two. Averaging a handful of readings gives a steadier number for free, and it teaches you that some noise can be filtered rather than fixed.
- Build a board from several markers. Four markers in a fixed arrangement let you compute where the camera is relative to the whole board, not just to one corner of it.
- Make the program refuse to guess. If consecutive readings jump more than a few millimetres, treat that as a lost marker rather than a moved one, and keep the last position instead of chasing the noise.
- Log every reading to a file with a timestamp. Run it for an hour and you will see exactly how much the numbers drift as the arm warms up.

MISSION DEBRIEF

1. The program stores distance in metres and prints it in millimetres. Why keep both?
2. The pump offsets are crossed over in the code. What would you check before deciding that was a bug and correcting it?
3. Your readings are roughly 3% too large. Which of `marker_size_m`, the calibration, or the lens distortion is the most likely cause, and how would you tell them apart?
4. Why does the program need both `rvecs` and `tvecs`? Name a task that would fail with only one of them.



steamoji™

MISSION

COMPLETE